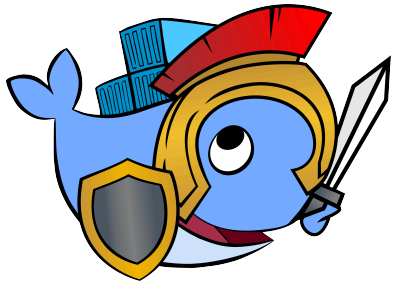


CONTAINER & KUBERNETES SECURITY

OWASP Switzerland Mini-Training – September 25th, 2024

$ whoami

- Sven Vetsch
- Redguard AG
  - Co-founder
  - Head of Innovation & Development
- Co-Lead OWASP Switzerland
- Working in information security for ~18 years

Contact: sven.vetsch@redguard.ch

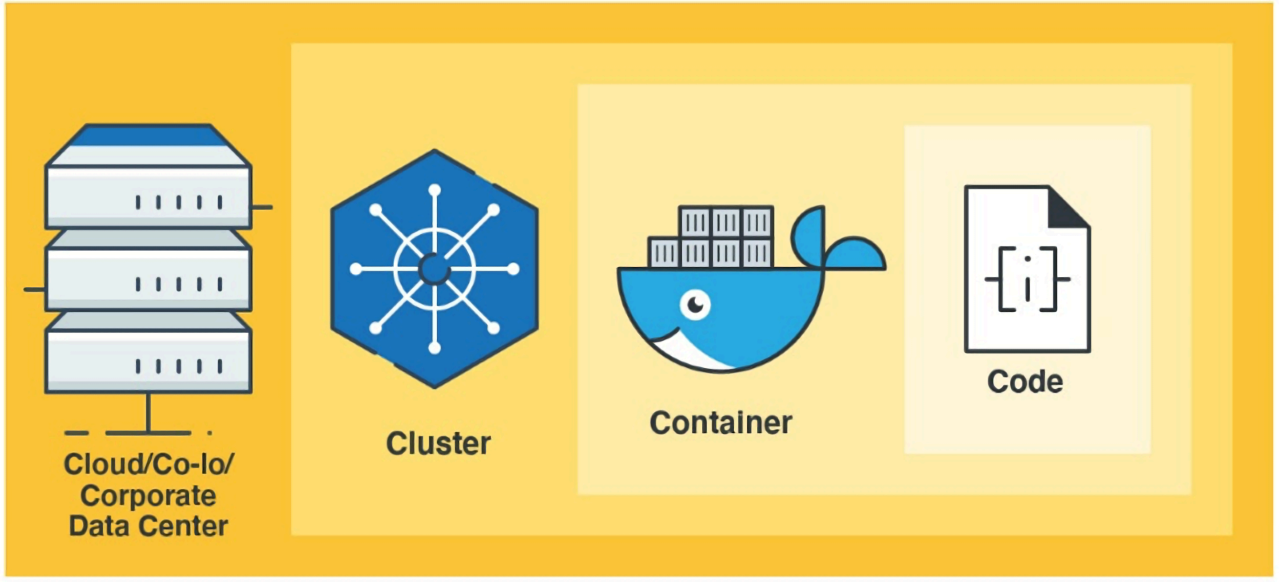
Disclaimer / Infos

- This presentation/training is an extract of Redguard's two day training shouldn’t be considered complete - it's a starting point
- Due to very limited time, there won't be hand-on labs.
- We'll look at some concepts with Docker that mostly also apply when using other container runtimes or when running containers in Kubernetes just to keep it simple.
- The training may showcases different tools but there are always alternatives so please perform a proper evaluation before start using them in your organization.

CLOUD NATIVE SECURITY

The big picture

4C's of Cloud Native security



Each layer of the Cloud Native security model builds/depends upon the next outermost layer.

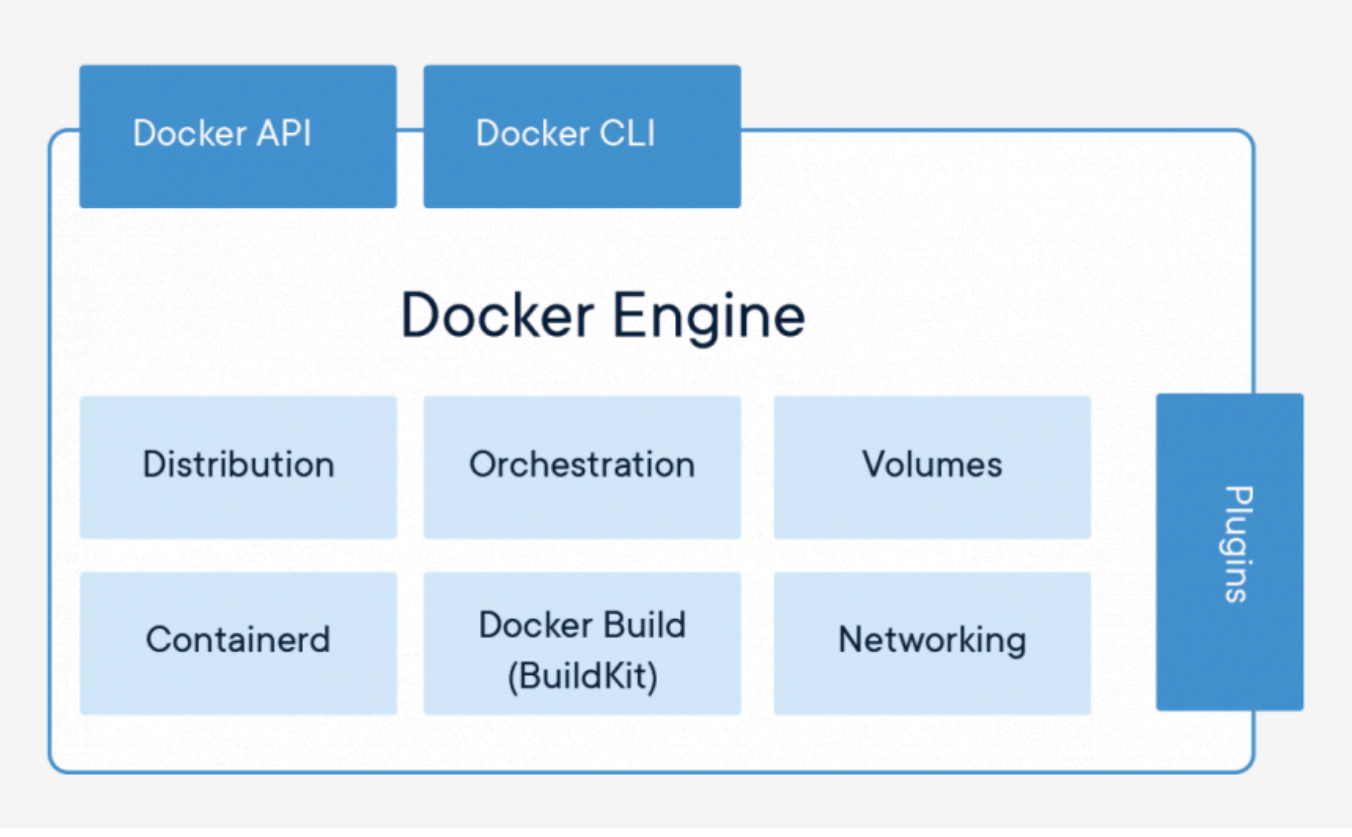
CONTAINER BASICS

(on the example of Docker)

Docker

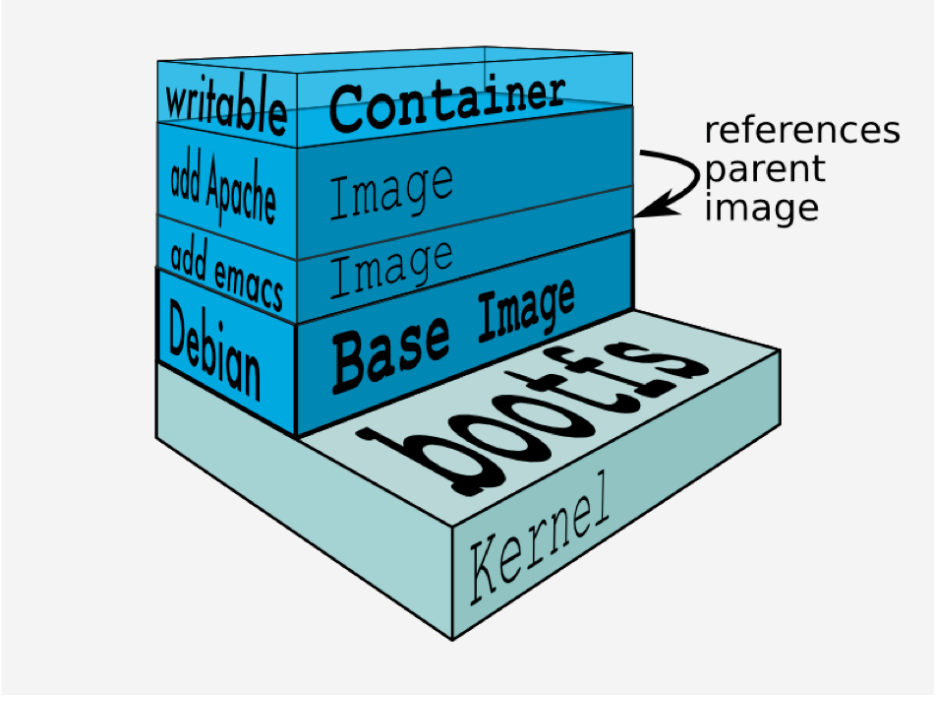
- Open platform for
  - developing
  - shipping
  - running
- Separates applications from infrastructure
- Manage infrastructure by managing applications
- Ability to package and run an application in a loosely isolated environment called a container
- Written in Go
- Not the only container runtime (but the most well known)

Docker Engine

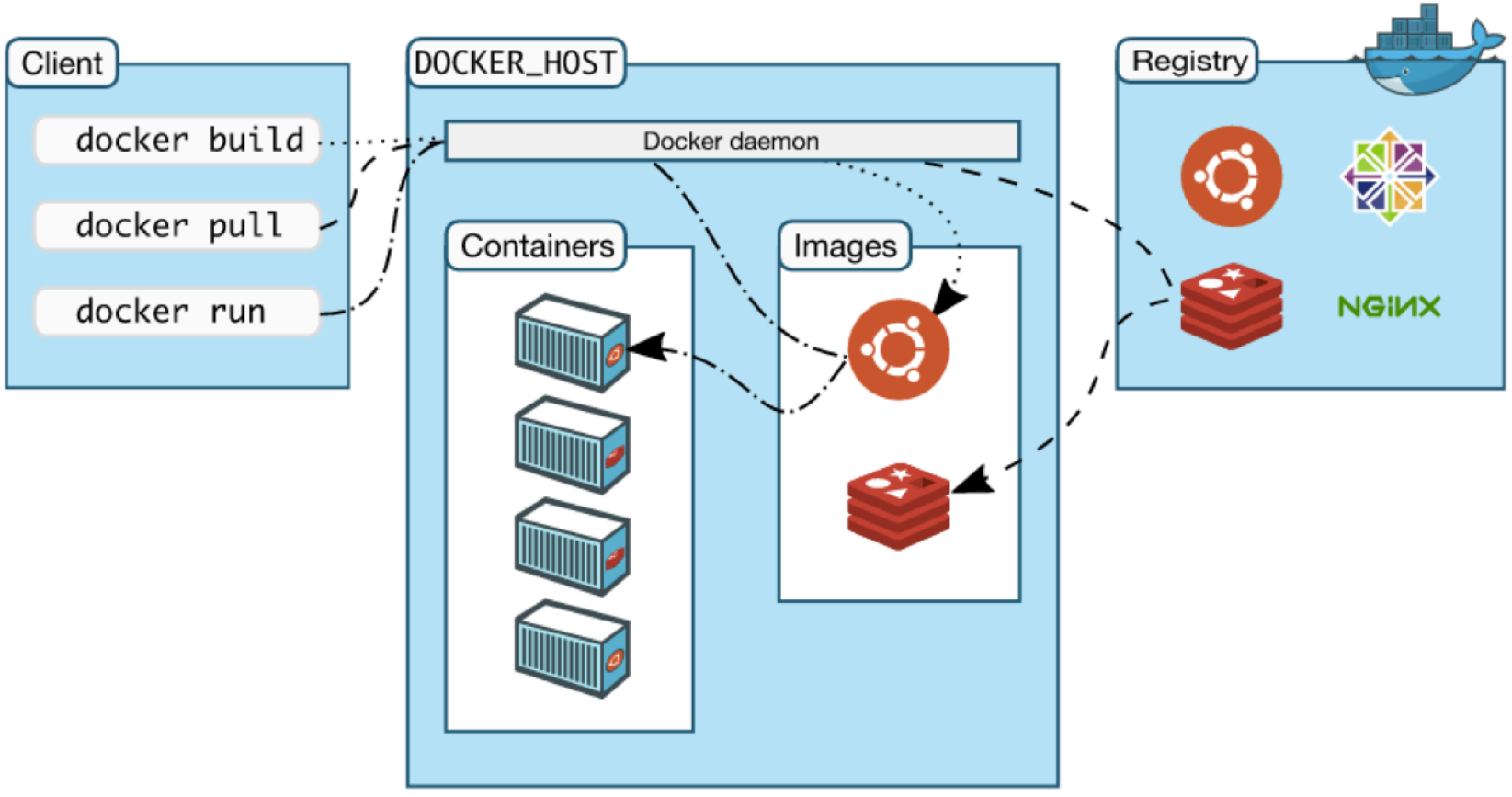


Images

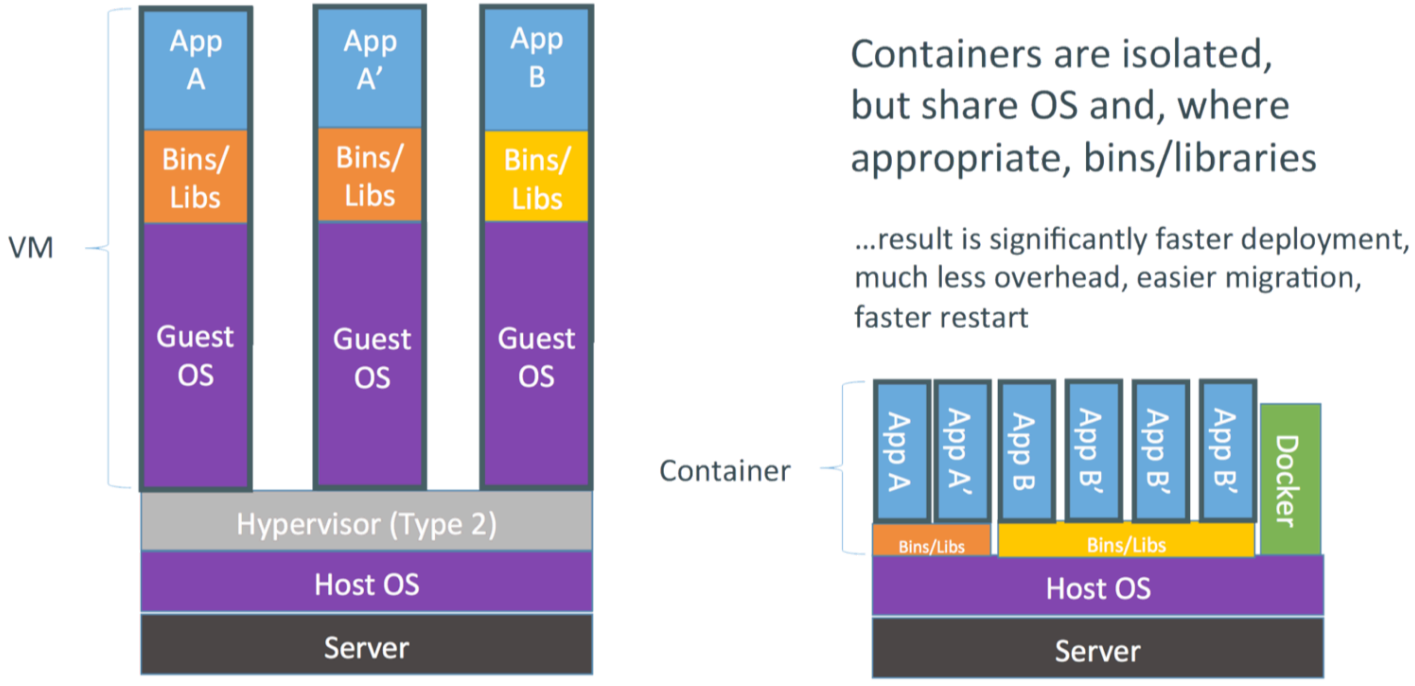
Images are the underlying building blocks of each container.



Docker Architecture



Containers vs. Virtual Machines



Containers vs. Virtual Machines

- Containers are **not** Virtual Machines (VMs)
- Containers leverage abstraction of the OS
- VMs leverage abstraction of hardware
- Containers are “run”, VMs have to boot

Containers

- Container == runnable instance of an image
- Extends a base/parent image
- Lightweight (kernel-based namespacing)
- Read-write layer on top of the image
- Copy-on-write (COW)

Containers

REDGUARD
SECURING YOUR ASSETS

Outside:

```
$ ps auxh | wc -l
177
```

Inside:

```
# ps aux
PID   USER     TIME   COMMAND
  1   root     0:00   sh
 11   root     0:00   ps aux
```

DOCKER/CONTAINER SECURITY RISKS

General Technical Risks

- Kernel Exploits
- Hardware Focused Attacks
- Container Separation / Breakouts
- Denial of Service (DoS)
- Poisoned Images
- Insecure data storage / volumes
- Non-Encrypted Communication
- Support Services (e.g. container registry, etcd, ...)
- ...

HARDENING & ATTACK SURFACE REDUCTION

Use trusted base/parent images

REDGUARD
SECURING YOUR ASSETS

- Official Images: Prioritize those from official software maintainers.
- Verified Sources: Only use images from trusted registries or publishers.
- Avoid 'Latest' Tag: Pin specific image versions or digests.
- Regular Updates: Keep base images up-to-date to address vulnerabilities.
- Vulnerability Scanning: Scan images before use and in your CI/CD pipeline.
- Consider Curated Registries: Leverage internal or trusted external registries.

Container Image Scanning Example

REDGUARD
SECURING YOUR ASSETS

- Sooner or later vulnerabilities will be identified in packages that you've got in your images.
- Automated vulnerability scanning is the key to keep a clear view on the current security status.

```
$ trivy image -s "LOW,MEDIUM,HIGH,CRITICAL" --ignore-unfixed redguard/lab

lab (debian 10.8)
=================================
Total: 37 (LOW: 4, MEDIUM: 6, HIGH: 22, CRITICAL: 5)
...

opt/bitnami/tomcat/webapps_default/ROOT.war (jar)
================================================
Total: 8 (LOW: 0, MEDIUM: 2, HIGH: 5, CRITICAL: 1)
```

PS: Trivy can also scan whole Kubernetes clusters.

.dockerignore File

Use a `.dockerignore` file to avoid having not needed things in your image.

Example `.dockerignore` file:

```
.git
.cache
.env*
src/
```

Restrict CPU, RAM, ...

Guarantees 50% of the available CPU power per second to the container:

```
$ docker run -it --rm --cpus=".5" ubuntu /bin/bash
```

Limit the memory (RAM) available to a container:

```
$ docker run -it --rm -m="512m" ubuntu /bin/bash
```

Limit the max. amount of processes running inside of the

```
$ docker run -it --rm --pids-limit 6 ubuntu /bin/bash
```

Mount root-FS read-only

```
$ docker run --rm -it \
   --read-only alpine \
  touch x

touch: x: Read-only file system
```

If you need to write in specific locations use mountpoints or just `--tmpfs /data` if you don't need persistance.

Don't overlook this possibility as it drastically increases the security of most systems/applications.

Non-root User

Dockerfile:

```
FROM alpine
RUN addgroup -S app && adduser -S -g app app
USER app
```

```
$ docker build -t redguard/non-root .
```

Now test which user is used inside of a container based on `redguard/non-root`

```
$ docker run --rm -it redguard/non-root whoami
app

$ docker run --rm -it --user root redguard/non-root whoami
root
```

Privileged Containers

REDGUARD
SECURING YOUR ASSETS

Don't use the `--privileged` flag unless you know exactly what you're doing!

(and even then it's likely not the best way of accomplishing what you want)

```
$ docker run --rm -it debian ls /dev/sd*
ls: cannot access '/dev/sda': No such file or directory
ls: cannot access '/dev/sda1': No such file or directory
ls: cannot access '/dev/sda2': No such file or directory
```

```
$ docker run --rm -it --privileged debian ls /dev/sd*
/dev/sda  /dev/sda1  /dev/sda2
```

Results in a **container breakout** when e.g. mounting the physical disk inside of the container.

root by using Docker

REDGUARD
SECURING YOUR ASSETS

We can bypass everything even simpler:

```
$ echo $(whoami)@$(hostname)
ubuntu@lab-server-39323432

$ docker run -it --rm \
    --privileged \
    --net=host --pid=host --ipc=host \
    --volume /:/host \
    busybox \
    chroot /host

$ echo $(whoami)@$(hostname)
root@lab-server-39323432
ubuntu
```

So remember: **Providing access to Docker means root on the host system.**

Capabilities (1/2)

- Capabilities are a set of syscalls used for administrative tasks.
- They are used to add/remove features (in the form of syscalls) to/from containers.
- Docker drops most most capabilities by default.
- A list of enabled by default capabilities in Docker can be found e.g. here.
- Want to read about all capabilities? Read the man page.

Capabilities (1/2)

- Capabilities are a set of syscalls used for administrative tasks.
- They are used to add/remove features (in the form of syscalls) to/from containers.
- Docker drops most most capabilities by default.
- A list of enabled by default capabilities in Docker can be found e.g. here.
- Want to read about all capabilities? Read the man page.

Example of dropping all capabilities:

```
$ docker run --rm -it \
  --cap-drop ALL \
  alpine ash
/ # touch x
/ # chown nobody x
chown: x: Operation not permitted
```

Capabilities (2/2)

Drop all capabilities but at the same time add *CAP_CHOWN* again.

```
$ docker run --rm -it \
    --cap-drop ALL --cap-add CHOWN \
  alpine ash
/ # touch x
/ # chown nobody x
/ # ls -lah x
-rw-r--r--    1 nobody   root           0 May  4 13:37 x
```

You can go even more granular by using Seccomp rules.

LOGGING & MONITORING

Logging

```
$ docker logs website
172.17.0.1 - - [12/March/2021:14:15:14 +0000] "GET /?shell=ls%20-lah HTT
172.17.0.1 - - [12/March/2021:14:15:22 +0000] "GET /?shell=echo%20%22%3C
```

```
$ dockerd --log-driver=syslog \
  --log-opt \
  syslog-address=udp://1.2.3.4
```

sysdig

> Sysdig is a simple tool for deep system visibility, with native support for containers. Think about sysdig as strace + tcpdump + htop + iftop + lsof + ...awesome sauce.

Start sysdig and have it output every directory the user `ubuntu` visits:

```
$ sudo apt install -y sysdig
$ sudo sysdig -p"%evt.arg.path" \
"evt.type=chdir and user.name=redguard"
```

```
$ whoami && pwd
  ubuntu
  /home/ubuntu
$ mkdir some-folder
$ cd some-folder
```

sysdig in Action with Containers

REDGUARD
SECURING YOUR ASSETS

```
sudo sysdig -w sysdig.scap container.name=alpine

sysdig -pc -r sysdig.scap \
-p"%user.name@%container.name (%container.id)
  > %proc.name %proc.args" \
evt.type=execve

root@alpine (f74848325fb7)
  > sh
root@alpine (f74848325fb7)
  > whoami
root@alpine (f74848325fb7)
  > sh
root@alpine (f74848325fb7)
  > touch x
```

Falco

> Falco is a behavioral activity monitor designed to detect anomalous activity in your applications. Powered by sysdig's system call capture infrastructure, Falco lets you continuously monitor and detect container, application, host, and network activity... all in one place, from one source of data, with one set of rules.

Examples on what you can detect:

- A shell is run inside a container
- A server process spawns a child process of an unexpected type
- Unexpected read of a sensitive file (like `/etc/shadow`)
- A non-device file is written to `/dev`
- A standard system binary (like `ls`) makes an outbound network connection
- ...

Falco Alerts

When Falco detects suspicious behavior, it sends alerts via one or more of the following channels:

- Writing to standard error
- Writing to a file
- Writing to syslog
- Pipe to a spawned program. A common use of this output type would be to send an email for every *falco* notification.

Run Falco

```
# falco
```

Or use Docker:

```
$ docker run -it --rm --privileged \
  -v /var/run/docker.sock:/host/var/run/docker.sock \
  -v /dev:/host/dev \
  -v /proc:/host/proc:ro \
  -v /boot:/host/boot:ro \
  -v /lib/modules:/host/lib/modules:ro \
  -v /usr:/host/usr:ro \
  falcosecurity/falco:latest
```

Run Falco

```
# falco
```

Or use Docker:

```
$ docker run -it --rm --privileged \
  -v /var/run/docker.sock:/host/var/run/docker.sock \
  -v /dev:/host/dev \
  -v /proc:/host/proc:ro \
  -v /boot:/host/boot:ro \
  -v /lib/modules:/host/lib/modules:ro \
  -v /usr:/host/usr:ro \
  falcosecurity/falco:latest
```

Let's trigger Falco:

```
$ docker run -it --rm falcosecurity/falco-event-generator
$ docker run -it --rm alpine cat /etc/shadow
```

Falco Rules

REDGUARD
SECURING YOUR ASSETS

```
- rule: shell_in_container
  desc: notice shell activity within a container
  condition: container.id != host and proc.name = bash
  output: shell in a container (user=%user.name container_id=%container.
  priority: WARNING
```

```
$ sudo falco -r rule.yaml
$ docker run --rm -it ubuntu bash
... Warning shell in a container (user=<NA> container_id=aa45a5e8c851
    container_name=funny_germain shell=bash parent=<NA> cmdline=bash)
```

Docker Diff

- `docker diff` shows changes to the filesystem of a container.
- It's a good way to see what a container (or an attacker inside it) has done.
- It's also a good way to see what your application does (e.g. for `--read-only`).

```
$ $ docker run --name diff-test alpine touch x
$ docker diff diff-test
A /x
```

KUBERNETES

What is Kubernetes?

To make it easy ...

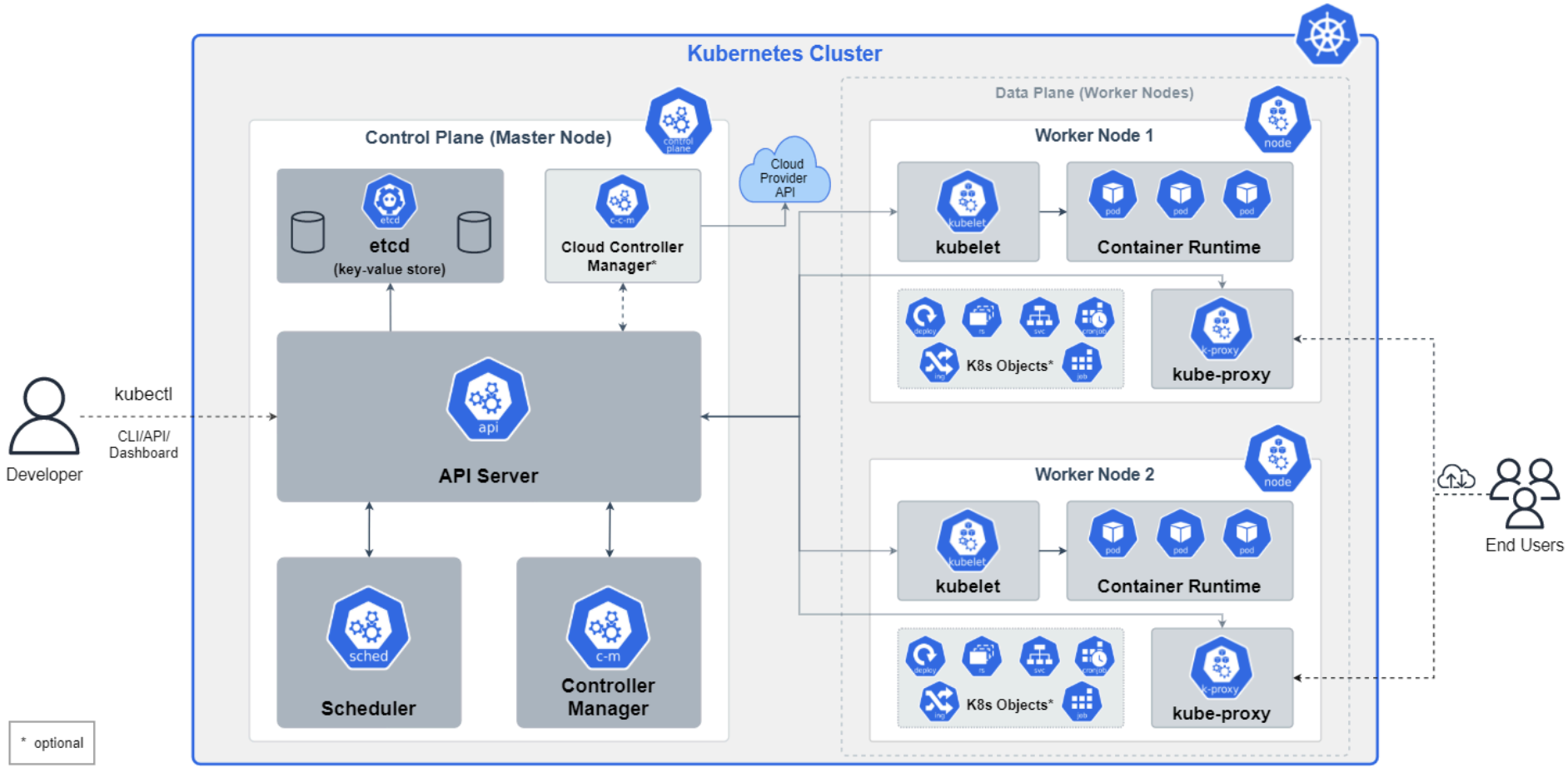
REDGUARD
SECURING YOUR ASSETS

Kubernetes

≅

General Purpose
Platform-as-a-Service (PaaS)
for containers

Kubernetes Archtitecture



ATTACK SURFACE

MITRE ATT&CK Threat Matrix

REDGUARD
SECURING YOUR ASSETS

A mapping to Kubernetes was done by Microsoft unfortunately with very limited information

Initial Access	Execution	Persistence	Privilege Escalation	Defense Evation	Credential Access	Discovery	Lateral Movement	Collection	Impact
Using Cloud Credentials	Exec into container	Backdoor container	Privileged container	Clear container logs	List K8s secrets	Access the K8s API server	Access cloud resources	Images from a private registry	Data destruction
Compromised images in registry	bash/cmd inside container	Writable hostPath mount	Cluster-admin binding	Delete K8s events	Mount service principal	Access Kubelet API	Container service account		Resource Hijacking
Kubeconfig file	New container	Kubernetes CronJob	hostPath mount	Pod / container name similarity	Access container service account	Network mapping	Cluster internal networking		Denial of service
Application vulnerability	Application exploit (RCE)	Malicious admission controller	Access cloud resource	Connect from proxy server	Applications credentials in configuration files	Access Kubernetes dashboard	Applications credentials in configuration files		
Exposed sensitive interfaces	Sidecar injection				Access managed identity credential	Instance metadata API	Writable volume mounts on the host		
					Malicious admission controller		CoreDNS poisoning		
							ARP poisoning and IP spoofing		

Blog: Secure containerized environments with updated threat matrix for Kubernetes

Redguard Kubernetes Threat Matrix

REDGUARD
SECURING YOUR ASSETS

Kubernetes Threat Matrix

Initial access	Execution	Persistence	Privilege escalation	Defense evasion	Credential access	Discovery	Lateral movement	Collection	Impact
Using Cloud	Exec into Container	Backdoor Container	Privileged Container	Clear Container Logs	List K8s secrets	Access the K8s API server	Access cloud resources	Images from a private repository	Data Destruction
Compromised images in registry	bash/cmd in container	Writable hostPath mount	Cluster-admin binding	Delete k8s events	Mount service principal	Access Kubelet API	Container service account		Ressource hijacking
Kubeconfig file	New container	Kubernetes CronJob	hostPath mount	Pod / container name similarity	Access container service account	Network mapping	Cluster internal networking		Denial of Service
Application vulnerability	Application exploit (RCE)	Malicious admission controller	Access cloud resources	Connect from proxy server	Applications credentials in configuration files	Access Kubernetes dashboard	Applications credentials in configuration files		
Exposed sensitive interfaces	SSH server running in inside container		Disable Namespacing		Access managed identity credentials	Instance metadata API	Writable volume mounts on the host		
	Sidecar injection				Malicious admission controller		CoreDNS poisoning		
							ARP poisoning and IP spoofing		

https://kubernetes-threat-matrix.redguard.ch/

Attack Demo

REDGUARD
SECURING YOUR ASSETS

Our attack path / kill chain:

Initial access	Execution	Persistence	Privilege escalation	Defense evasion	Credential access	Discovery	Lateral movement	Collection	Impact
Using Cloud	Exec into Container	Backdoor Container	Privileged Container	Clear Container Logs	List K8s secrets	Access the K8s API server	Access cloud resources	Images from a private repository	Data Destruction
Compromised images in registry	bash/cmd in container	Writable hostPath mount	Cluster-admin binding	Delete k8s events	Mount service principal	Access Kubelet API	Container service account		Ressource hijacking
Kubeconfig file	New container	Kubernetes CronJob	hostPath mount	Pod / container name similarity	Access container service account	Network mapping	Cluster internal networking		Denial of Service
Application vulnerability	Application exploit (RCE)	Malicious admission controller	Access cloud resources	Connect from proxy server	Applications credentials in configuration files	Access Kubernetes dashboard	Applications credentials in configuration files		
Exposed sensitive interfaces	SSH server running in inside container		Disable Namespacing		Access managed identity credentials	Instance metadata API	Writable volume mounts on the host		
	Sidecar injection				Malicious admission controller		CoreDNS poisoning		
							ARP poisoning and IP spoofing		

DEV(SEC)OPS CULTURE

It's not just about tech

The Organization

REDGUARD
SECURING YOUR ASSETS

- The most common "issue"
- Just calling something Dev(Sec)Ops doesn't magically make it Dev(Sec)Ops
- Many companies currently only have 1-2 people who can handle Kubernetes
  - It's the same people that also have to take care of CI/CD pipelines and all the tooling
  - They also are the ones planning the architecture and setting up the systems
  - No quality control and if this single person leaves the company ... have fun
- DevSecOps doesn't mean that you had three people (Dev, Sec & Ops) and now one person can do all of that on his/her own. It's about bringing people closer together and getting rid of unnecessary boundaries.
- Responsibilities aren't clear (e.g. who defines what an acceptable firewall rule is?)
- We'll leave this here for now as we want to talk about Kubernetes but keep in mind that most of the time the current state of the organization is a key problem.

ARCHITECTURE

Architecture

There are two general architecture philosphies:

1. One big cluster (per environment) for everything
2. A lot of small clusters e.g. one for every single application

We've seen both and it all boils down to your Kubernetes distribution and tooling.

For example if you're using Red Hat OpenShift you likely have a very limited amount of clusters but when using VMware Tanzu you likely have a lot more.

My recommendation: Go with big clusters but consider breaking them up depending on the use-case. For example having a cluster in your DMZ but also one which is strictly internal. Or also PROD, QA, DEV, ...

MAINTENANCE

Maintenance

- New (minor) version every four months (was three months before)
  - Too fast for many organizations.
  - This doesn't yet include all (security) patch releases
- Not enough experience nor automation to risk it.
- Updates must be done both on Kubernetes and the underlying nodes (idealy just replace them).
- Consider having *[Insert Favorite Cloud Provider]* maintaining your Kubernetes cluster(s)

HANDLING OF CONTAINERS & IMAGES

Handling of Containers & Images

REDGUARD
SECURING YOUR ASSETS

- It's the foundation. If the containers (including the applications running inside them) are insecure your Kubernetes cluster is at risk too.
- Hardening (examples):
  - Applications are secure on their own and packages up-to-date
  - **No** `privileged: true`
  - `automountServiceAccountToken: false`
  - Running as non-root user
  - Read-only root filesystem
- Internal *Cloud Native (Security) Guidelines*
- Vulnerability scanning of container images
  - One of the main things security teams want to have but most of them don't monitor the status once it is in place and they also don't enforce any restrictions.

Pod (Resource) Limits

REDGUARD
SECURING YOUR ASSETS

- Limiting CPU und memory consumption
- Makes sure that pods/containers can't cause a denial of service situation.

```
apiVersion: v1
kind: Pod
metadata:
  name: example-pod
spec:
  containers:
  - name: example-container
    image: nginx
    resources:
      requests: # minimum required
        cpu: 500m
        memory: 1Gi
      limits:   # maximum allowed
        cpu: 1000m
        memory: 2Gi
```

Security Context

REDGUARD
SECURING YOUR ASSETS

Configuring the pod's security context allows you to do various things:

- Specify the user (the user's ID) under which the process in the container will run.
- Prevent the container from running as root.
- Run the container in privileged mode, giving it full access to the node's kernel.
- Configure fine-grained privileges, by adding or dropping capabilities.
- Prevent the process from writing to the container's filesystem.

We've already seen the concepts with Docker so we'll focus on the behavior in k8s.

Security Context

REDGUARD
SECURING YOUR ASSETS

At least you should use:

- `AllowPrivilegeEscalation = false`
- `ReadOnlyRootFileSystem = true`
- `RunAsNonRoot = true`

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: example-pod
spec:
  containers:
  - name: example-container
    image: nginx
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      runAsNonRoot: true
      # privileged: false
```

(This Pod won't be able to start)

Security Context (runAsUser)

REDGUARD
SECURING YOUR ASSETS

```
kubectl run pod-with-defaults --image alpine --restart Never -- /bin/sleep 999999
kubectl exec pod-with-defaults -- id
# uid=0(root) gid=0(root) groups=...


cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: pod-as-user-guest
spec:
  containers:
  - name: main
    image: alpine
    command: ["/bin/sleep", "999999"]
    securityContext:
      runAsUser: 405
EOF

kubectl exec pod-as-user-guest -- id
# uid=405(guest) gid=100(users)
```

Security Context (runAsNonRoot)

REDGUARD
SECURING YOUR ASSETS

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: pod-run-as-non-root
spec:
  containers:
  - name: main
    image: alpine
    command: ["/bin/sleep", "999999"]
    securityContext:
      runAsNonRoot: true
EOF
```

```
kubectl get pod pod-run-as-non-root
# NAME                  READY   STATUS
# pod-run-as-non-root   0/1     CreateContainerConfigError

kubectl describe pod pod-run-as-non-root
# ... Error: container has runAsNonRoot and image will run as root ...
```

SEGREGATION

There's more than namespaces

Segregation

- Namespaces (just an organizational segregation mechanism)
- Permissions / Accessibility (RBAC)
- Data / Storage
- CPU / Memory
- Network
  - Nearly everywhere we've seen that any pod/container can still reach the kube-api and e.g. etcd which shouldn't be the case.
- Nodes (Taints / Tolerations)
  - Most of the time companies just make master nodes unschdulable but don't leverage the feature any further.
  - Even with taints one can e.g. schedule on master nodes by using `nodeName: my-master-node-1` (in `spec`).

Bypass Container Namespacing

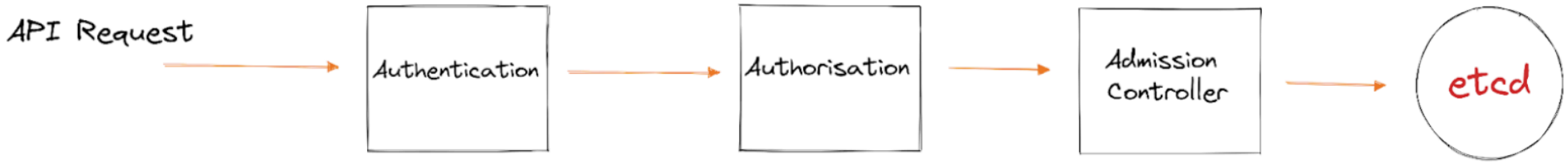
REDGUARD
SECURING YOUR ASSETS

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: noderootpod
spec:
  hostNetwork: true
  hostPID: true
  hostIPC: true
  containers:
  - name: noderootpod
    image: busybox
    securityContext:
      privileged: true
    volumeMounts:
    - mountPath: /host
      name: noderoot
    command: [ "/bin/sh", "-c", "--" ]
    args: [ "while true; do sleep 30; done;" ]
  volumes:
  - name: noderoot
    hostPath:
      path: /
EOF
```

Role-based access control (RBAC)

REDGUARD
SECURING YOUR ASSETS

Authentication is about knowing who someone is, authorization is about knowing what the authenticated entity is allowed to do.



Kubernetes offers various *Authentication Plugins*:

- X509 Client Certs
- Bearer token passed in an HTTP header
- OpenID Connect Tokens
- ...

Role-based access control (RBAC)

REDGUARD
SECURING YOUR ASSETS

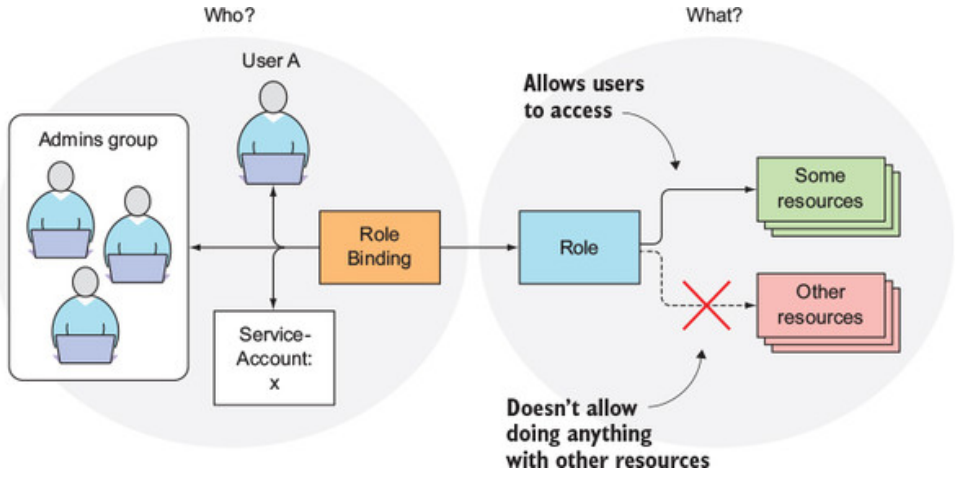
Kubernetes knows two different kinds of authenticated entities:

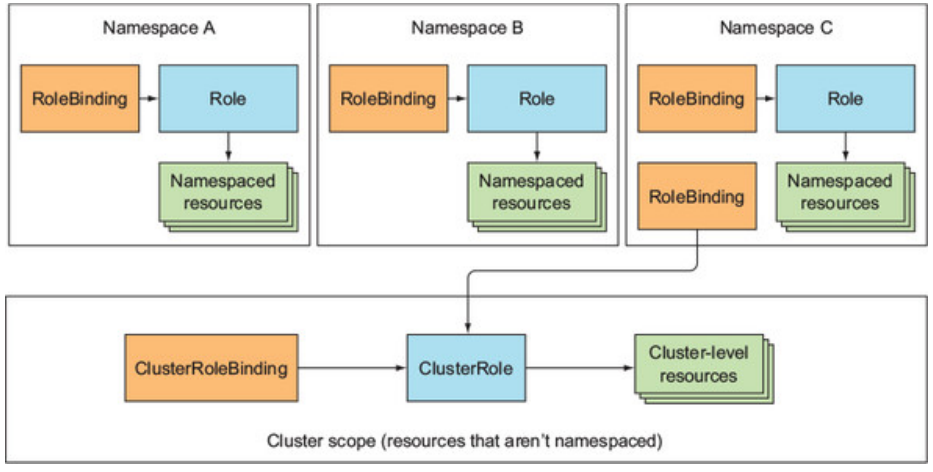
- **User**
  - For humans
  - Managed by external systems (like an ActiveDirectory)
  - You therefore can't create, update, or delete users through the k8s API server.
- **Service Account**
  - For machines
  - Managed as objects inside Kubernetes
  - Allows pods to get access to (parts of) the k8s API.

To better manage permissions (i.e. allowed actions on resources) there are *(Cluster) Roles*. Both users and service accounts can belong to one or more roles.

Role-based access control (RBAC)

To combine users/SAs with groups, Kubernetes uses *(Cluster) Role Bindings*.





Role-based access control (RBAC)

REDGUARD
SECURING YOUR ASSETS

```
kubectl create namespace rbac
kubectl create serviceaccount rbac-sa --namespace rbac

cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: rbac-pod
  namespace: rbac
spec:
  serviceAccountName: rbac-sa
  containers:
  - name: main
    image: curlimages/curl
    command: ["sleep", "9999999"]
  - name: ambassador
    image: luksa/kubectl-proxy:1.6.2
EOF

kubectl -n rbac wait pod rbac-pod --for=condition=Ready

kubectl -n rbac exec -ti rbac-pod -c main -- curl localhost:8001/api/v1/pods
kubectl -n rbac exec -ti rbac-pod -c main -- curl localhost:8001/api/v1/namespaces/rbac/pods
```

Role-based access control (RBAC)

REDGUARD
SECURING YOUR ASSETS

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: rbac
  name: pod-reader
rules:
- apiGroups: [""]
  verbs: ["get", "list"]
  resources: ["pods"]
EOF

# Or if you'd like to type less:
# kubectl create role pod-reader --verb=get --verb=list --resource=pods -n rbac
```

Role-based access control (RBAC)

REDGUARD
SECURING YOUR ASSETS

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: rbac
  name: pod-reader
rules:
- apiGroups: [""]
  verbs: ["get", "list"]
  resources: ["pods"]
EOF

# Or if you'd like to type less:
# kubectl create role pod-reader --verb=get --verb=list --resource=pods -n rbac
```

```
kubectl create rolebinding rbac-test --role=pod-reader --serviceaccount=rbac:rbac-sa -n rbac
```

Role-based access control (RBAC)

REDGUARD
SECURING YOUR ASSETS

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: rbac
  name: pod-reader
rules:
- apiGroups: [""]
  verbs: ["get", "list"]
  resources: ["pods"]
EOF

# Or if you'd like to type less:
# kubectl create role pod-reader --verb=get --verb=list --resource=pods -n rbac
```

```
kubectl create rolebinding rbac-test --role=pod-reader --serviceaccount=rbac:rbac-sa -n rbac
```

```
kubectl -n rbac exec -ti rbac-pod -c main -- curl localhost:8001/api/v1/pods
kubectl -n rbac exec -ti rbac-pod -c main -- curl localhost:8001/api/v1/namespaces/rbac/pods
```

Role-based access control (RBAC)

REDGUARD
SECURING YOUR ASSETS

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: rbac
  name: pod-reader
rules:
- apiGroups: [""]
  verbs: ["get", "list"]
  resources: ["pods"]
EOF

# Or if you'd like to type less:
# kubectl create role pod-reader --verb=get --verb=list --resource=pods -n rbac
```

```
kubectl create rolebinding rbac-test --role=pod-reader --serviceaccount=rbac:rbac-sa -n rbac
```

```
kubectl -n rbac exec -ti rbac-pod -c main -- curl localhost:8001/api/v1/pods
kubectl -n rbac exec -ti rbac-pod -c main -- curl localhost:8001/api/v1/namespaces/rbac/pods
```

```
# Grant access for all namespaces (cluster-wide):
kubectl create clusterrole clusterwide-pod-reader --verb=get,list --resource=pods
kubectl create clusterrolebinding clusterwide-test --clusterrole=clusterwide-pod-reader --serviceaccount=rbac:rbac-sa
kubectl -n rbac exec -ti rbac-pod -c main -- curl localhost:8001/api/v1/pods
```

Network Security

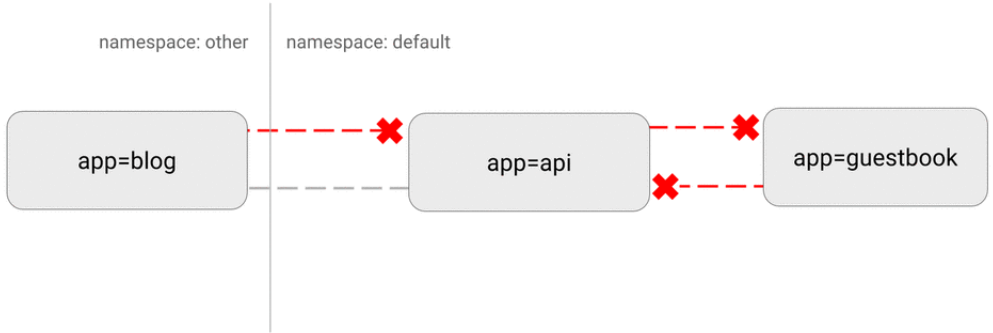
REDGUARD
SECURING YOUR ASSETS

- By default there's one flat network (every pod can reach everything)
  - This includes the kube-api, etcd, ...
  - In "old" networks we e.g. also don't want a web-app frontend to be able to directly talk to a database.
- Use network policies.
- Container Network Interface (CNI) plugins like *Cilium* can provide a lot more granular policies than the normal *NetworkPolicy*
- A Service Mesh like *Istio* can add better visibility and at the same time increase security (e.g. mTLS between pods)
- Keep in mind that old network attacks like IP spoofing or ARP poisoning in general still work in Kubernetes.

Network Policy Examples

REDGUARD
SECURING YOUR ASSETS

DENY all non-whitelisted traffic

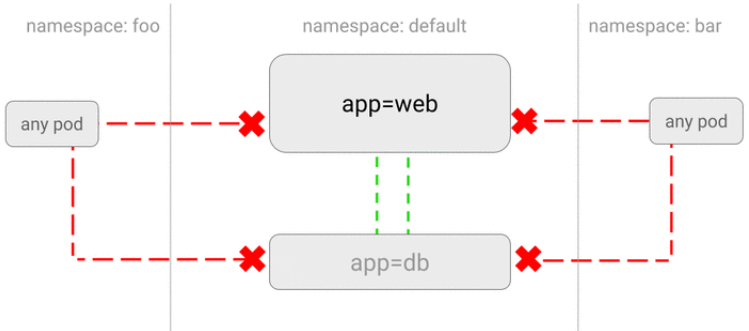



```
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  namespace: default
  name: default-deny-all
spec:
  podSelector: {}
  ingress: []
```

{} : Selecting All Pods / [] : Null Selector

Network Policy Examples

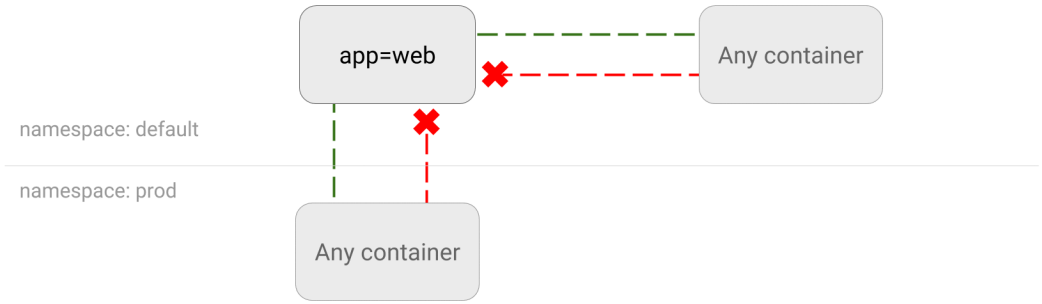
DENY all traffic from other namespaces




```
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  namespace: default
  name: deny-from-other-namespaces
spec:
  podSelector:
    matchLabels:
  ingress:
  - from:
    - podSelector: {}
```

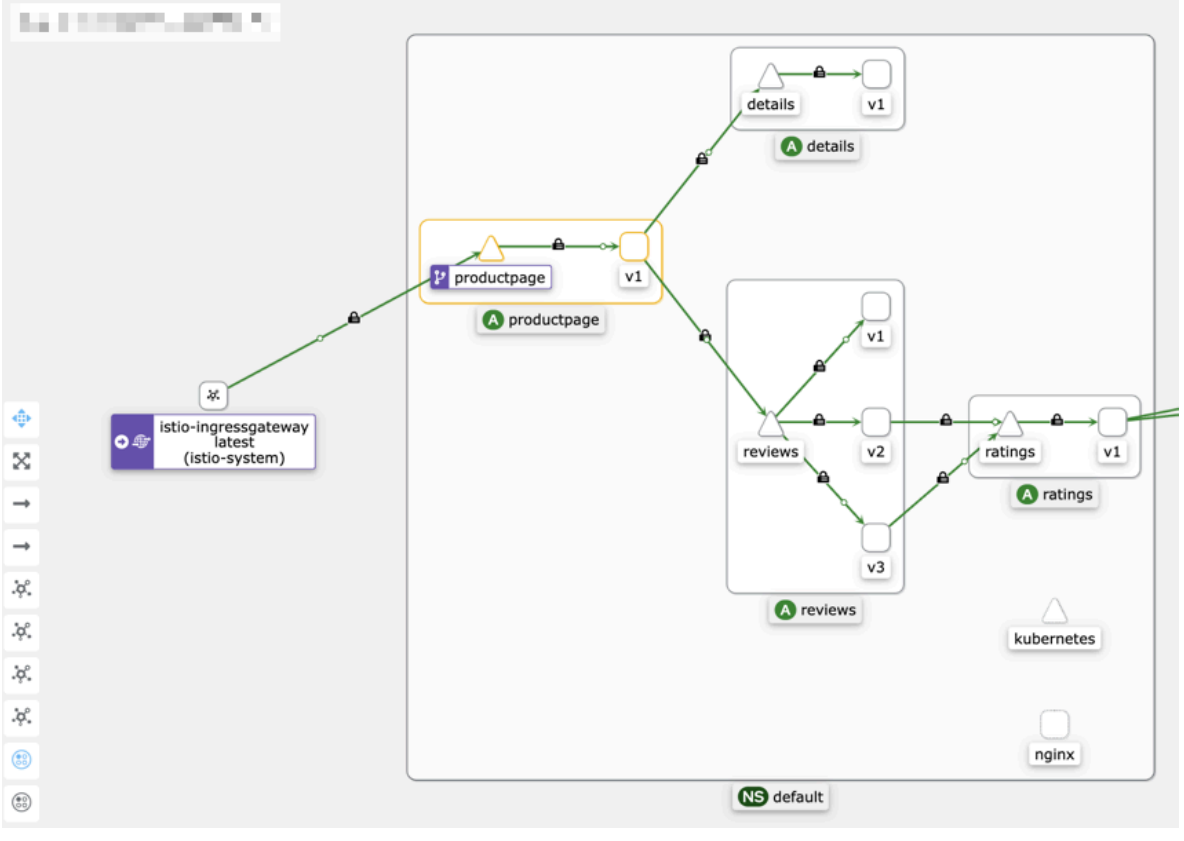
Network Policy Examples

DENY all traffic to an application




```
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  namespace: default
  name: web-deny-all
spec:
  podSelector:
    matchLabels:
      app: web
  ingress: []
```

Service Mesh

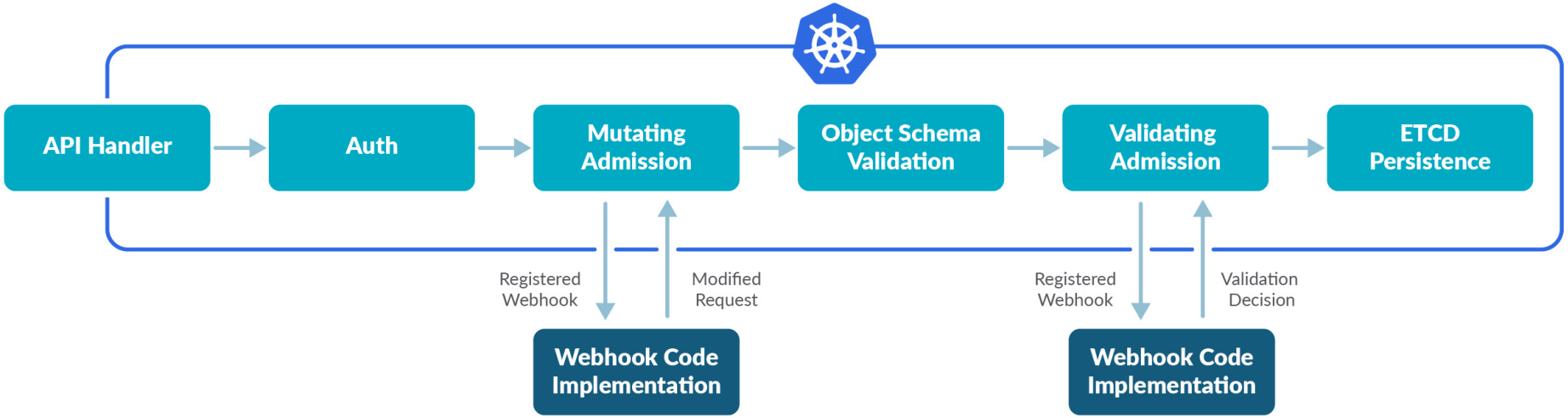


A screenshot from Istio with kiali

ADMISSION CONTROLLER

Policy as Code in Kubernetes

Admission Controller



- Basically one of the main security layers of Kubernetes
- Most used/known admission controller was *PodSecurityPolicy* (PSP)
  - Removed in Kubernetes v1.25
  - Replaced by *Pod Security Admission* (PSA) / *Pod Security Standards* (PSS)

Validating Admission Controller

REDGUARD
SECURING YOUR ASSETS

- Either accepts or rejects a request (aka. admission).
- For example, block all pods that request to run as root.
- Responsible for gating the API requests and enforcing custom policies prior to request execution or persistence of the new object in the etcd.
- Kubernetes forwards the JSON-encoded object that is about to be persisted in etcd to the the admission controller.
- Exceptions are possible.
- One object can be validated against multiple admission policy controls.
- Keep in mind that having too many restrictions can be frustrating for people who need to deploy their objects to Kubernetes.

Mutating Admission Controller

REDGUARD
SECURING YOUR ASSETS

- Help to automate the policy compliance processes.
- Very similar to *Validating Admission Controllers* but instead modifies the object and returns the result for further processing / persistance to Kubernetes.
- Validation could also be implemented in a *Mutating Admission Controller* but it's not recommended by any means.
- Mutating objects is very powerful and has a great potential for auto-remediation but ending up with a different object in your cluster than the one initialy sent to it can be confusing.

Pod Security Standards

REDGUARD
SECURING YOUR ASSETS

There are three different profiles for the Pod Security Standards:

Privileged: Unrestricted policy, providing the widest possible level of permissions. (DANGER ZONE)

Baseline: Minimally restrictive policy which prevents known privilege escalations.

Restricted: Heavily restricted policy, following current Pod hardening best practices.

```
$ kubectl create ns pss-demo
namespace/pss-demo created
$ kubectl label ns pss-demo 'pod-security.kubernetes.io/enforce=baseline'
namespace/pss-demo labeled
$ kubectl get ns pss-demo --show-labels
NAME       STATUS   AGE    LABELS
pss-demo   Active   2m9s   ...,pod-security.kubernetes.io/enforce=baseline
```

Pod Security Standards

REDGUARD
SECURING YOUR ASSETS

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: privileged-pod
  namespace: pss-demo
spec:
  containers:
  - name: privileged
    image: nginx
    securityContext:
      privileged: true
EOF
```

Pod Security Standards

REDGUARD
SECURING YOUR ASSETS

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: privileged-pod
  namespace: pss-demo
spec:
  containers:
  - name: privileged
    image: nginx
    securityContext:
      privileged: true
EOF
```

```
... pods "privileged-pod" is forbidden: violates PodSecurity "baseline:latest": privileged
  (container "privileged" must not set securityContext.privileged=true)
```

Open Policy Agent (OPA)

REDGUARD
SECURING YOUR ASSETS

- OPA Gatekeeper - Policy Controller for Kubernetes

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredLabels
metadata:
  name: all-must-have-owner
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Namespace"]
  parameters:
    message: "All namespaces must have an `owner` label"
    labels:
      - key: owner
        allowedRegex: "^[a-zA-Z]+.example.demo$"
```

Open Policy Agent (OPA)

REDGUARD
SECURING YOUR ASSETS

Example in plain Rego:

```
package kubernetes.admission

deny[msg] {
    input.request.kind.kind == "Pod"
    some i
    image := input.request.object.spec.containers[i].image
    not startswith(image, "example.com/")
    msg := sprintf("image '%v' comes from untrusted registry", [image])
}
```

Open Policy Agent (OPA)

REDGUARD
SECURING YOUR ASSETS

Example in plain Rego:

```
package kubernetes.admission

deny[msg] {
    input.request.kind.kind == "Pod"
    some i
    image := input.request.object.spec.containers[i].image
    not startswith(image, "example.com/")
    msg := sprintf("image '%v' comes from untrusted registry", [image])
}
```

There are of course alternatives to OPA (like e.g. *Kyverno*).

OPA Gatekeeper

REDGUARD
SECURING YOUR ASSETS

Easy explanation on what it is:

> OPA directly running inside of k8s by providing all the needed components (e.g. admission controller) and introduces specific CRDs.

Installation:

```
kubectl apply -f https://raw.githubusercontent.com/open-policy-agent/gatekeeper/master/deploy/gatekeeper.yaml
```

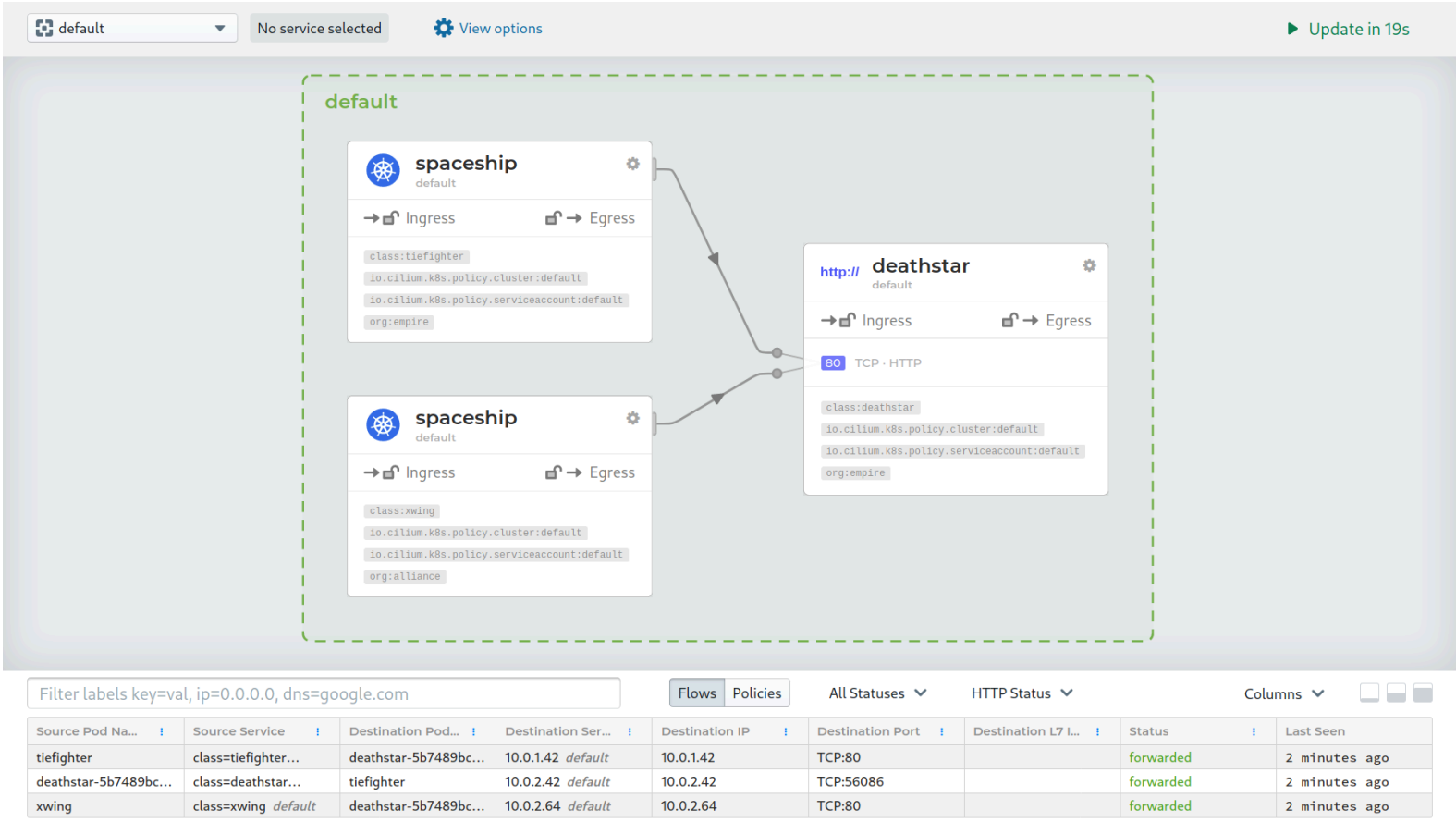
Instead of a complex setup we can look at the very good official demo/example:

```
git clone https://github.com/open-policy-agent/gatekeeper.git
cd ./gatekeeper/demo/basic
./demo.sh
```

MONITORING AND VISIBILITY

Know what's going on

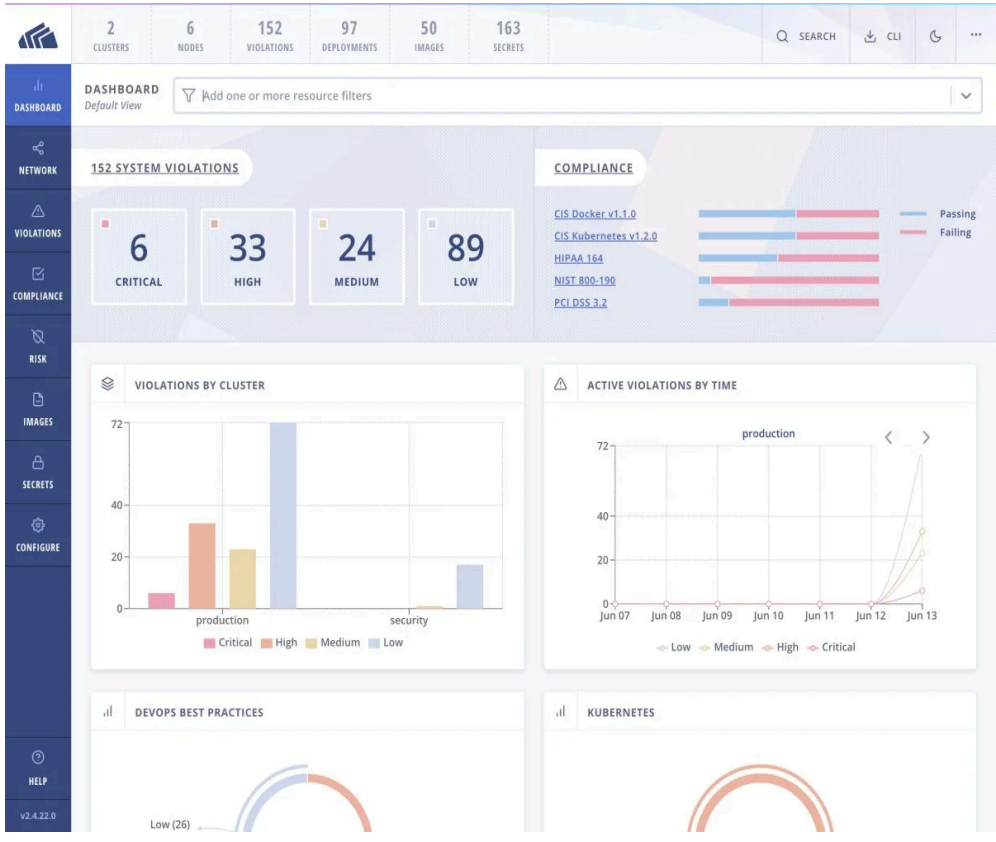
Network (Policies)



Cilium Service Map & Hubble UI

Cluster / Pods / Containers

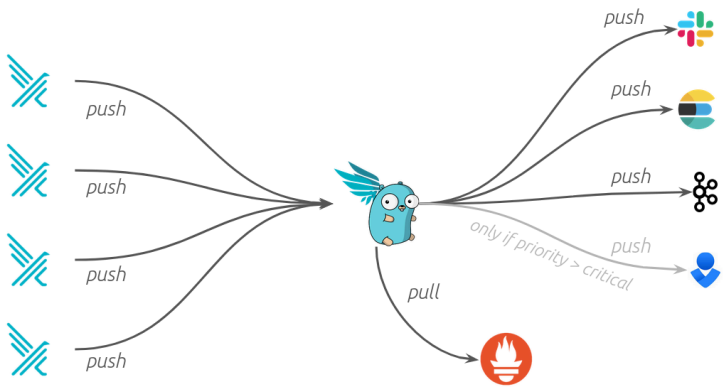
- Prometheus / Grafana
- Elasticsearch / Fluentd / Kibana (EFK)
- Splunk
- StackRox
- Portshift
- dynatrace
- OpenShift
- Sysdig
- Starboard
- Octant
- Lens
- ...



Falco (yes also in k8s)

- Monitoring all the way down to Linux syscalls if you like.
- Simple anomaly detection due to containers.
- Generates alerts when a treat is detected
- Simple but powerfull rule engine

IMHO the one must-have security monitoring tool for Kubernetes.
(Even more so in combination with *Falcosidekick*)



Audit Log

- The component to log (security relevant) events in Kubernetes (or more specific the Kubernetes API)
- Can basically catch everything
- Not enabled by default
- Needs tuning to get the "right amount" of information
  - Else you can expect Gigabytes of audit logs per day
- Is very valuable in case of an incident.
  - Or even just to do normal debugging of the cluster.

Audit Log

REDGUARD
SECURING YOUR ASSETS

```
mkdir -p ~/.minikube/files/etc/ssl/certs
cat <<EOF > ~/.minikube/files/etc/ssl/certs/audit-policy.yaml
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
  - level: Metadata
    namespaces: ["default"]
    verbs: ["create", "get","list","delete"]
    resources:
      - group: ""
        resources: ["pods"]
EOF

minikube start \
  --extra-config=apiserver.audit-policy-file=/etc/ssl/certs/audit-policy.yaml \
  --extra-config=apiserver.audit-log-path=-

kubectl run nginx --image nginx

kubectl logs --tail 20 kube-apiserver-minikube -n kube-system
```

```
{"kind":"Event",...,"requestURI":"/api/v1/namespaces/default/pods?fieldManager=kubectl-run","verb":"create",
"user":{"username":"minikube-user","groups":["system:masters","system:authenticated"]},
"sourceIPs":["192.168.49.1"]...,"objectRef":{"resource":"pods","namespace":"default","name":"nginx","apiVersion":"v1"},
"responseStatus":{"metadata":{},"code":201},...,"annotations":{..."authorization.k8s.io/decision":"allow",...}}
```

Sorry, but Minikube forces us to do it kind of hacky

Q & A

What more do you want to know
about container / k8s security?

Thank you for your attention